

Architecture Finale : Système de Métriques JobbingTrack

[← Retour à la documentation](#) | [← README principal](#) |  [Navigation](#)

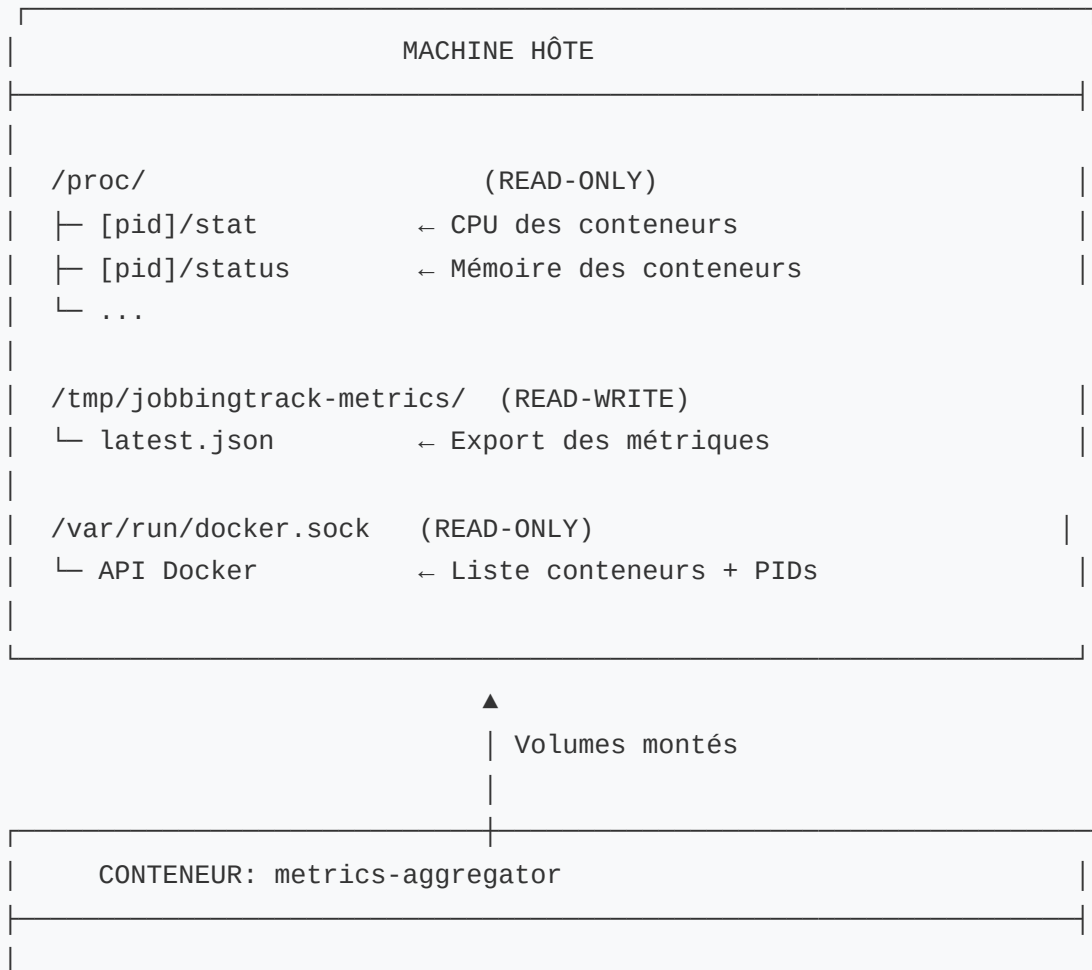
 [Dépannage Métriques](#)

Objectif

Créer un système de collecte de métriques **sécurisé et performant** avec :

-  **Lecture seule** des données système (`/proc` en read-only)
-  **Collecte native** via Docker API + `/proc` de l'hôte
-  **Export JSON** vers `/tmp` pour partage avec le frontend
-  **Aucune écriture** sur la machine hôte (sauf `/tmp`)

Architecture



LECTURE

- └ /host/proc/ ← Mount de /proc (read-only)
- └ /var/run/docker.sock ← Docker API (read-only)

ÉCRITURE

- └ /tmp/metrics/ ← Export JSON (write vers hôte)

PROCESSUS

- | | |
|---|-------------|
| 1. Docker API: Lister conteneurs
→ Obtenir PIDs des conteneurs | |
| 2. Pour chaque conteneur: | |
| - Lire /host/proc/[pid]/stat | ← READ-ONLY |
| - Lire /host/proc/[pid]/status | ← READ-ONLY |
| - Calculer CPU et Mémoire | |
| 3. Agréger les métriques: | |
| - Tous conteneurs (system) | |
| - Conteneurs JobbingTrack | |
| 4. Exporter vers /tmp/metrics/
latest.json | ← WRITE |

API HTTP (optionnel)

- └ :3014/api/v1/metrics



FRONTEND (Next.js)

- OPTION 1: Lire /tmp/jobbingtrack-metrics/latest.json
- └ Mode fichier (si accessible via volume partagé)

OPTION 2: HTTP API

- └ GET http://localhost:3014/api/v1/metrics

AFFICHAGE

- └ Vue d'ensemble: CPU/Mémoire système
- └ Conteneurs: Liste avec détails individuels

Configuration docker-compose.yml

```
jobbingtrack-metrics-aggregator:
  image: jobbingtrack-metrics-aggregator
  container_name: jobbingtrack-metrics-aggregator
  ports:
    - "3014:3014"
  volumes:
    # READ-ONLY: Socket Docker pour lister conteneurs
    - /var/run/docker.sock:/var/run/docker.sock:ro

    # READ-ONLY: /proc de l'hôte pour lire les stats
    - /proc:/host/proc:ro

    # READ-WRITE: Dossier d'export des métriques
    - /tmp/jobbingtrack-metrics:/tmp/metrics:rw
  networks:
    - jobbingtrack-network
  restart: unless-stopped
```

Exemple: Collecte pour `jobbingtrack-api-gateway`

Étape 1: Obtenir le PID

```
// Via Docker API
const container = docker.getContainer('jobbingtrack-api-gateway')
const inspect = await container.inspect()
const pid = inspect.State.Pid // Ex: 12345
```

Étape 2: Lire CPU depuis `/host/proc/12345/stat`

```
const statData = await fs.readFile('/host/proc/12345/stat', 'utf8')
const statFields = statData.split(' ')
const utime = parseInt(statFields[13]) // Temps CPU utilisateur
const stime = parseInt(statFields[14]) // Temps CPU système
const totalTime = utime + stime
```

Étape 3: Lire Mémoire depuis `/host/proc/12345/status`

```
const statusData = await fs.readFile('/host/proc/12345/status', 'utf8')
const vmrssMatch = statusData.match(/VmRSS:\s+(\d+)\s+kB/)
const memoryKB = parseInt(vmrssMatch[1])
const memoryMB = Math.round(memoryKB / 1024)
```

Étape 4: Construire l'objet métrique

```
{
  "jobbingtrack-api-gateway": {
    "pid": 12345,
    "cpu": {
      "usage": 2.5,
      "percentage": 2.5,
      "totalTime": 150000
    },
    "memory": {
      "usage": 256,          // MB
      "limit": 512,         // MB
      "percentage": 50.0
    },
    "status": "running"
  }
}
```

Étape 5: Exporter vers `/tmp/metrics/latest.json`

```
const metricsData = {
  containers: { /* tous les conteneurs */ },
  system: {
    containersAggregate: {
      cpu: { percent: "3.5", containers: 16 },
      memory: { percent: "25.0", used: 2048, limit: 8192 }
    }
  },
  timestamp: "2025-10-29T12:00:00.000Z"
}

await fs.writeFile('/tmp/metrics/latest.json', JSON.stringify(metricsData, null, 2))
```

Structure du Fichier Exporté

/tmp/jobbingtrack-metrics/latest.json :

```
{
  "containers": {
    "jobbingtrack-api-gateway": {
      "pid": 12345,
      "cpu": {
        "usage": 2.5,
        "percentage": 2.5
      },
      "memory": {
        "usage": 256,
        "limit": 512,
        "percentage": 50.0
      },
      "status": "running"
    },
    "jobbingtrack-frontend": { /* ... */ },
    "jobbingtrack-postgres": { /* ... */ }
  },
  "system": {
    "cpu": {
      "percent": 1.5,
      "cores": 16,
      "model": "Intel i7"
    },
    "memory": {
      "total": 15989,
      "used": 2332,
      "percent": 14.58
    },
    "containersAggregate": {
      "cpu": {
        "percent": "3.25",
        "total": "52.0",
        "containers": 16
      },
      "memory": {
        "used": 2048,
        "limit": 8192,
        "percent": "25.0"
      }
    },
    "jobbingtrack": {
      "containers": {
        "count": 8,
        "cpu": {
          "averagePercent": 15,
          "totalPercent": 120
        }
      },

```

```

    "memory": {
      "used": 1024,
      "limit": 4096,
      "percent": 25
    }
  },
  "timestamp": "2025-10-29T12:00:00.000Z"
}

```

Sécurité

✓ Permissions Read-Only

- `/proc` monté en **read-only** (`:ro`)
- `/var/run/docker.sock` en **read-only** (`:ro`)
- **Aucune modification** du système hôte possible

✓ Isolation

- Le conteneur ne peut **pas écrire** dans `/proc`
- Le conteneur ne peut **pas modifier** les conteneurs Docker
- Export limité à `/tmp/jobbingtrack-metrics`

✓ Principe du Moindre Privilège

- Pas de `privileged: true`
- Pas de `--cap-add`
- Accès minimal requis

Déploiement

Créer le dossier d'export

```

mkdir -p /tmp/jobbingtrack-metrics
chmod 777 /tmp/jobbingtrack-metrics

```

Construire et démarrer

```
cd /home/pactivisme/Documents/Dev/Person/JobbingTrack
docker build -t jobbingtrack-metrics-aggregator backend/metrics-aggregator-ser
docker-compose up -d jobbingtrack-metrics-aggregator
```

Vérifier l'export

```
# Attendre quelques secondes
sleep 10

# Vérifier le fichier
cat /tmp/jobbingtrack-metrics/latest.json | jq .

# Vérifier les logs
docker logs jobbingtrack-metrics-aggregator | grep EXPORT
```



Utilisation dans le Frontend

Option 1: Lire le fichier JSON directement

```
// Si le frontend a accès au filesystem de l'hôte
const metrics = JSON.parse(
  fs.readFileSync('/tmp/jobbingtrack-metrics/latest.json', 'utf8')
)
```

Option 2: Via API HTTP

```
// API standard
const response = await fetch('http://localhost:3014/api/v1/metrics')
const metrics = await response.json()
```

Option 3: WebSocket (temps réel)

```
const ws = new WebSocket('ws://localhost:3014')
ws.on('metrics-update', (data) => {
```

```
console.log('Nouvelles métriques:', data)
})
```

✓ Avantages de cette Architecture

1. **Sécurisé** : Read-only sur toutes les sources critiques
2. **Performant** : Lecture directe de `/proc` , pas d'intermédiaire
3. **Découplé** : Frontend peut lire `/tmp` sans API
4. **Fiable** : Pas de dépendances externes (cAdvisor, Prometheus)
5. **Simple** : Un seul service à gérer
6. **Temps réel** : Collecte toutes les 10 secondes
7. **Persistant** : Données exportées survivent aux redémarrages

↻ Cycle de Collecte

1. Toutes les 10 secondes (cron)
 - ↓
2. Docker API → Liste des conteneurs + PIDs
 - ↓
3. Pour chaque PID:
 - Lire `/host/proc/[pid]/stat`
 - Lire `/host/proc/[pid]/status`
 - ↓
4. Calculer métriques agrégées
 - ↓
5. Exporter vers `/tmp/metrics/latest.json`
 - ↓
6. Broadcast WebSocket (optionnel)
 - ↓
7. Servir via HTTP API (optionnel)

📝 Commandes Utiles

```
# Voir les métriques en temps réel
watch -n 2 'cat /tmp/jobbingtrack-metrics/latest.json | jq .system.containers/'

# Filtrer un conteneur spécifique
cat /tmp/jobbingtrack-metrics/latest.json | jq '.containers["jobbingtrack-api-'
```



```
# Compter les conteneurs
cat /tmp/jobbingtrack-metrics/latest.json | jq '.containers | length'

# Voir les logs de collecte
docker logs -f jobbingtrack-metrics-aggregator | grep -E "PROC|EXPORT"
```

Résumé

Avant : 3 services (simple-metrics, system-metrics, aggregator) + cAdvisor

Après : 1 service (aggregator) utilisant `/proc` natif

Collecte : `/proc` de l'hôte (read-only)

Export : `/tmp/jobbingtrack-metrics/latest.json` (read-write)

Consommation : Frontend lit le fichier JSON ou utilise l'API HTTP